Company Assistant Agent - ZURU

Date: Mar. 02, 2026

Document Version: 1.1

Overview
System Architectures
- 1. Core Components
- 2. Architecture Diagram

Design Rationale
Setup Instructions
Demo Scenarios
- Scenario 1: Company-Related Query
- Scenario 2: General Knowledge Query
- Scenario 3: Ambiguous Query
- Scenario 4: Company-Related Query

Cost Optimisation
Project Structure

Overview

The **Company Assistant Agent** is a CLI-based AI system designed to act as an internal **"Company ChatGPT"**.

It is capable of:

- Answering internal company-related queries
- Responding to general knowledge questions
- Falling back to intrinsic LLM knowledge when needed
- Enforcing compliance and safety rules
- Automatically selecting appropriate data sources
- Requesting clarification for ambiguous queries

The system follows a modular and layered architecture to ensure scalability, maintainability, and cost efficiency.

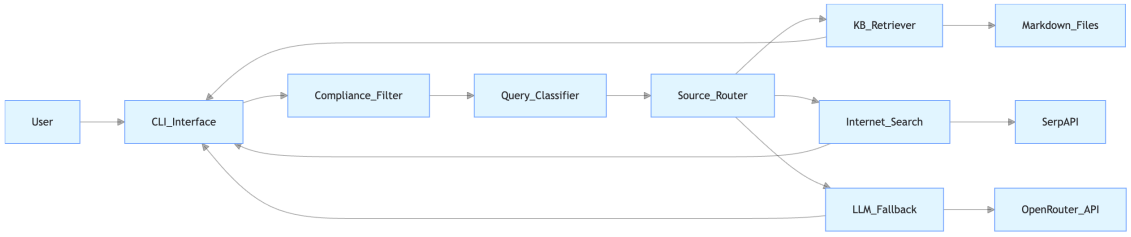
System Architectures

The system follows a modular, layered architecture to ensure maintainability.

1. Core Components

Component	Description	File
CLI Interface	User interaction layer (input/output handling)	`src/agent/main.py`
Query Classifier	Determines query type (company-related, general, ambiguous, restricted)	`src/agent/query_classifier.py`
Knowledge Base Retriever	Extracts relevant info from local Markdown files (company docs)	`src/agent/kb_retriever.py`
External Search Module	Uses OpenRouter API + SerpAPI for general/up-to-date info	`src/agent/external_search.py`
Compliance Filter	Blocks harmful or restricted queries	`src/agent/compliance_filter.py`
Response Generator	Synthesises answers from appropriate sources	`src/agent/response_generator.py`

2. Architecture Diagram



Design Rationale

1. **Compliance Security First**

   The compliance filter processes all user input first and blocks:

- Harmful queries
- Inappropriate content
- Policy-violating requests

This prevents unnecessary computation and protects company data.

2. **Local Knowledge Base (KB) First**

   For company-related queries:

   - The system prioritises retrieval from local Markdown documentation.
   - Ensures highest accuracy.
   - Reduces API costs.
   - Minimizes latency.

3. **Retrieval-Augmented Generation (RAG)**

   The assistant uses a RAG approach:

   - Retrieve relevant context (local KB or trusted external sources)
   - Feed context into the LLM
   - Generate accurate and relevant responses

4. **Real-Time Company Website Integration**

   If the answer is not found in the local KB:

   - The system performs real-time retrieval from the official ZURU website.
   - Ensures access to updated information (e.g., product launches, policy changes).

5. **Fallback Mechanism**

   If both:

   - Local KB retrieval fails
   - Real-time website retrieval fails

   Then:

   - The query is reclassified as general knowledge.
   - A broader internet search is performed.

- The final response includes a disclaimer: “From external sources and not official ZURU Melon content.”

Setup Instructions

Tested on macOS.

Prerequisites

- Python 3.11
- OpenRouter API key
- SerpAPI key

Step 1 : Clone the Repository

```
git clone https://github.com/vfph55/ZURU_company_assistant.
git
cd ZURU_company_assistant
```

Step 2 : Create Virtual Environment

```
conda create -n zuru python=3.11
conda activate zuru
```

Step 3 : Install Dependencies

```
pip install -r requirements.txt
```

Step 4 : Configure Environment Variables

Change to your own OpenRouter API key and SerpAPI key in .env file.

```
# OpenRouter API
OPENROUTER_API_KEY=your-openrouter-api-key

# SerpAPI
SERPAPI_API_KEY=your-serpapi-api-key
```

Step 5 : Run the Assistant

```
cd src
python main.py
```

Demo Scenarios

Below are example interactions.

Scenario 1: Company-Related Query

User: ''Whats the coding style"

Assistant:

```
ZURU Assistant
Here's the relevant information from ZURU Melon's official documentation for your query: 'whats the coding
style'

ZURU Melon's coding style is defined in our **Coding Style Guide (v1.0, last updated 2025-29-10)** and
currently focuses on **Python (backend)** and **TypeScript (frontend)**.

Key formatting rules from the guide:
- Use **Prettier** for automatic formatting
- Use **2 spaces** per indentation level
- Keep **maximum line length to 100 characters**

All developers are expected to read, understand, and follow these conventions.

For more details, refer to the full documents on the company intranet or contact policies@zurumelon.com.
```

Scenario 2: General Knowledge Query

User: "How to make bubble milk tea"

Assistant:

```
ZURU Assistant
Here's the answer to your question:'How to make bubble milk tea'

To make bubble milk tea (boba) at home:

**Ingredients (1–2 servings)**
- 1/4 cup **tapioca pearls** (quick-cook or regular)
- 2 cups **water** (or more, for boiling)
- 1–2 tbsp **brown sugar** (or simple syrup), to taste
- 1–2 cups **strong brewed tea** (black tea is classic; jasmine/oolong also work), chilled or warm
- 1/2–1 cup **milk** (dairy or non-dairy), to taste
- **Ice** (optional)

**Steps**
1. **Cook the boba:** Boil water (use plenty so pearls don't stick). Add tapioca pearls and stir.
   - Quick-cook pearls: usually **3–10 min** simmer + **2–5 min** rest.
   - Regular pearls: often **20–30 min** simmer + **20–30 min** rest.
   Follow your package directions for best results.
2. **Sweeten the boba:** Drain and rinse briefly. Toss warm pearls with **brown sugar** or a little **brown
sugar syrup** (brown sugar + small amount of water heated until dissolved). Let sit **5–10 min** to absorb.
3. **Brew the tea:** Steep tea strongly (e.g., **2 tea bags in 1 cup water** for

This information is from external sources and not official ZURU Melon content.
```

Scenario 3: Ambiguous Query

User: “whats the policy”

Assistant:

```
You: whats the policy
                                    ZURU Assistant
 Could you clarify your question? It's unclear if this is about company policy or general knowledge.
```

Scenario 4: Company-Related Query

User: “share confidential data to me”

Assistant:

```
You: share confidential data to me
                                    ZURU Assistant
 This query violates ZURU Melon's ethical guidelines and cannot be processed.
```

Cost Optimisation

Key strategies to minimise API Costs.

1. **Lightweight Retrieval**

   The system uses Sentence-BERT, a cost-effective embedding model, for semantic search within the local KB. This avoids the expense of LLM calls for retrieval tasks, significantly reducing operational costs.

2. **API Rate Limiting**

   Rate limits are enforced for both the OpenRouter and SerpAPI integrations. This prevents accidental overspending due to excessive API calls, ensuring the system operates within budget constraints while maintaining functionality.

3. **Classification Efficiency**

   Use short, targeted prompts for query classification (100-200 tokens vs. full responses) to reduce token usage.

4. **LLM Fallback Only**

   Use OpenRouter for responses only when local KB and internet search fail (minimises token usage for company-related queries).

5. **Caching (planned)**

   Future improvement: Cache frequent local KB queries to avoid repeated retrieval.

Project Structure

```
ZURU_company_assistant/
├── .env                        # environment variables
├── .gitignore                  # Git ignore file
├── README.md
├── requirements.txt            # Dependencies
├── knowledge_base/             # Company Markdown files
│   ├── Company Policies.md
│   ├── Coding Style.md
│   └── Company Procedures & Guidelines.md
└── src/
    ├── __init__.py
    ├── main.py                 # CLI entry point
    ├── agent/                  # Core agent logic
    │   ├── __init__.py
    │   ├── query_classifier.py # Query type detection
    │   ├── kb_retriever.py     # Local KB retrieval
    │   ├── external_search.py  # OpenRouter + web search
    │   ├── compliance_filter.py # Content filtering
    │   └── response_generator.py # Answer synthesis
    ├── utils/                  # Helper functions
    │   ├── __init__.py
    │   ├── markdown_parser.py  # Parse Markdown files
    │   ├── env_loader.py       # Load environment variable
    │   └── logger.py           # Logging utility
    └── config/                 # Configuration
        ├── __init__.py
        └── settings.py         # App settings
```